

Distributed Systems

(4th edition, version 01)

<https://www.distributed-systems.net/>

Chapter 01: Introduction

Brief Computing History

1945: modern computer era begins

1945-1985: computers large and expensive

mid-1980s:

- powerful microprocessors
- invention of high-speed computer networks

Brief Computing History - Networks

Local-area networks (LANs)

Allow thousands of machines within building to be connected

Wide-area network (WANs)

Allow hundreds of millions of machines all over the earth to be connected

Speeds varying from tens of thousands to hundreds of millions bps and more.

Brief Computing History - Networks

Period	Development
1960s	Packet-switching and computer networking concepts emerge
1969	ARPANET → early computer WAN
Early 1970s	Ethernet developed → early LAN
1970s	LAN and WAN technologies rapidly mature
1980s	Ethernet LANs become widespread; TCP/IP becomes dominant
1990s	Internet/WAN connectivity becomes widespread commercially

Brief Computing History

Now: miniaturization of computer systems, with perhaps the smartphone as the most impressive outcome.

The size of a networked computer system may vary from a handful of devices, to millions of computers.

Most networked computer systems can be accessed from anywhere in the world because they are connected to Internet.

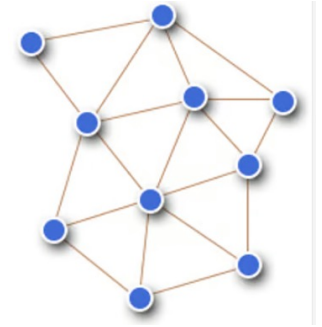
Alternative approach

Two definitions

- A **decentralized system** is a networked computer system in which processes and resources are **necessarily** spread across multiple computers.
- A **distributed system** is a networked computer system in which processes and resources are **sufficiently** spread across multiple computers.

Distributed vs Centralized

Distributed system uses multiple computers in a network
Each node has its own memory

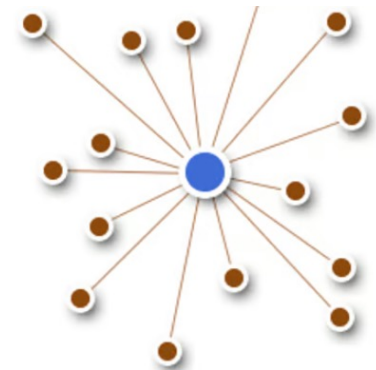


Centralized system follows client-server architecture built around single, powerful server.

Less powerful nodes submit requests to central server

Easier to manage

But...entire system depends on central server

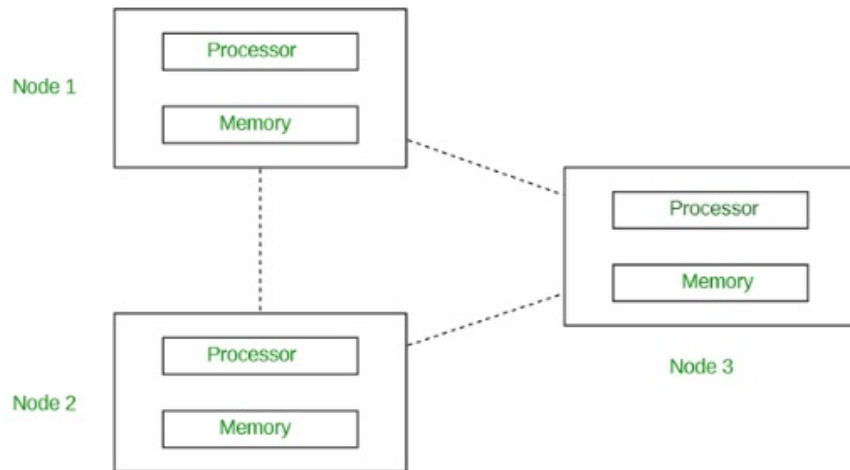


Distributed vs Parallel

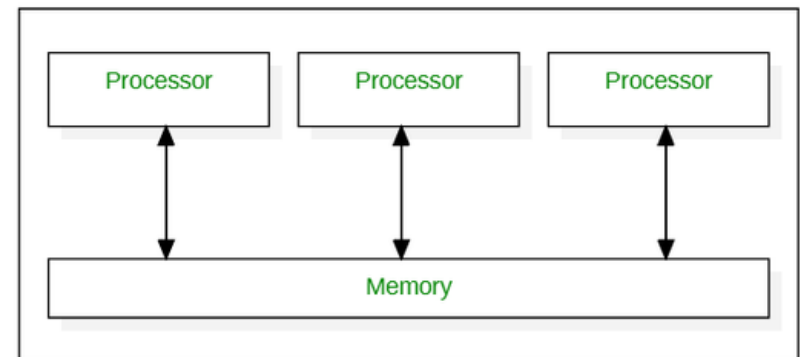
Distributed computing uses multiple computers in a network
Each node has its own memory

Parallel computing usually uses one computer with multiple processors
There is a single memory unit

Distributed Computing



Parallel Computing



Source: <https://www.geeksforgeeks.org/different-computing-paradigms/>

Distributed System – Example 1

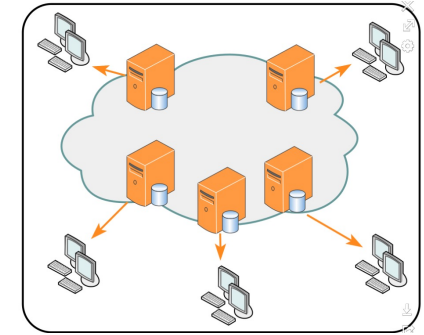
Gmail

Close to 1.8 billion users as of 2025

Seems to use two servers: inbox and outbox

Behind the scenes, the entire Google Mail service has been implemented and spread across many computers

Distributed System – Example 2



Source: Wikipedia

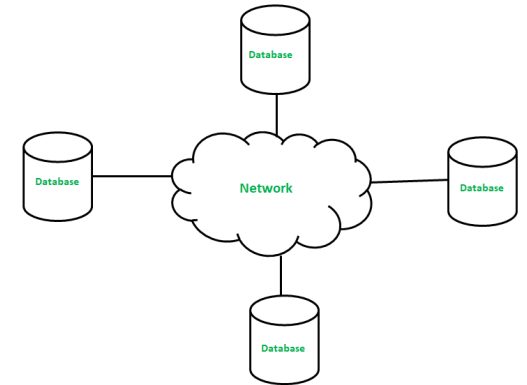
Content Delivery Networks (CDNs)

Geographically distributed network of servers and their data centers

Content copied and spread across various servers of the CDN

When visiting a website, user is redirected to nearby server that holds relevant content.

Distributed System – Example 3



Distributed Database (e.g., Amazon DynamoDB)

Scalable: Automatically distributes data and traffic across multiple servers to handle varying throughput and storage requirements.

High Availability and Durability:

Data automatically replicated across multiple Availability Zones within an AWS Region.

Flexibility and Data Model: DynamoDB supports a flexible NoSQL data model, accommodating both key-value and document data.

Perspectives on distributed systems

Distributed systems are complex. Key considerations include:

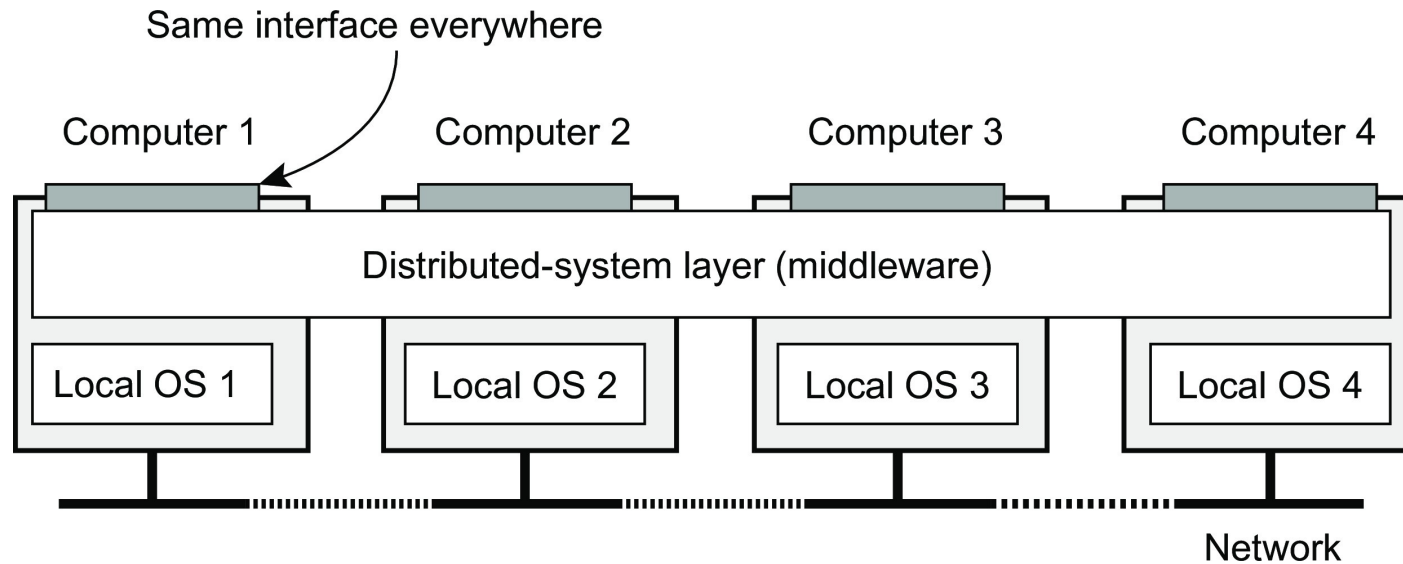
- **Communication**: facilities for exchanging data
- **Coordination**: application-independent algorithms
- **Naming**: how do you identify resources?
- **Consistency** and **replication**
- **Fault tolerance**: keep running in the presence of partial failures
- **Security**: ensure authorized access to resources

What do we want to achieve?

Overall design goals

- Support sharing of resources
- Distribution transparency – a bit of a confusing term, means that users don't realize the system is distributed (details are abstracted away)
- Openness – includes extensibility
- Scalability

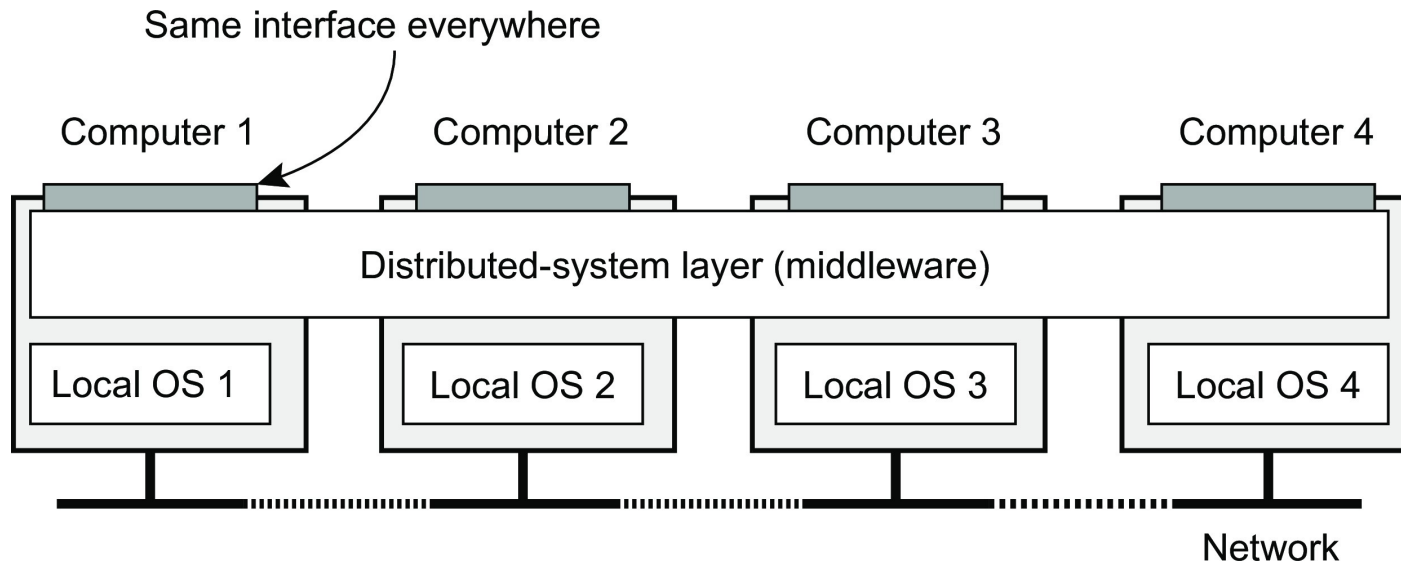
Distribution transparency



What is transparency?

*The phenomenon by which a distributed system attempts to **hide** the fact that its processes and resources are **physically distributed across multiple computers**, possibly **separated by large distances**.*

Distribution transparency



What is transparency?

*The phenomenon by which a distributed system attempts to **hide** the fact that its processes and resources are **physically distributed across multiple computers**, possibly **separated by large distances**.*

Observation

Distribution transparency is handled through many different techniques in a layer between applications and operating systems: a **middleware layer**

Scale in distributed systems

Observation

Many developers of modern distributed systems easily use the adjective “scalable” without making clear **why** their system actually scales.

Scale in distributed systems

Observation

Many developers of modern distributed systems easily use the adjective “scalable” without making clear **why** their system actually scales.

At least three components

- Number of users or processes (**size scalability**)
- Maximum distance between nodes (**geographical scalability**)
- Number of administrative domains (**administrative scalability**)

Size scalability

Root causes for scalability problems with centralized solutions

- The computational capacity, limited by the CPUs
- The storage capacity, including the transfer rate between CPUs and disks
- The network between the user and the centralized service

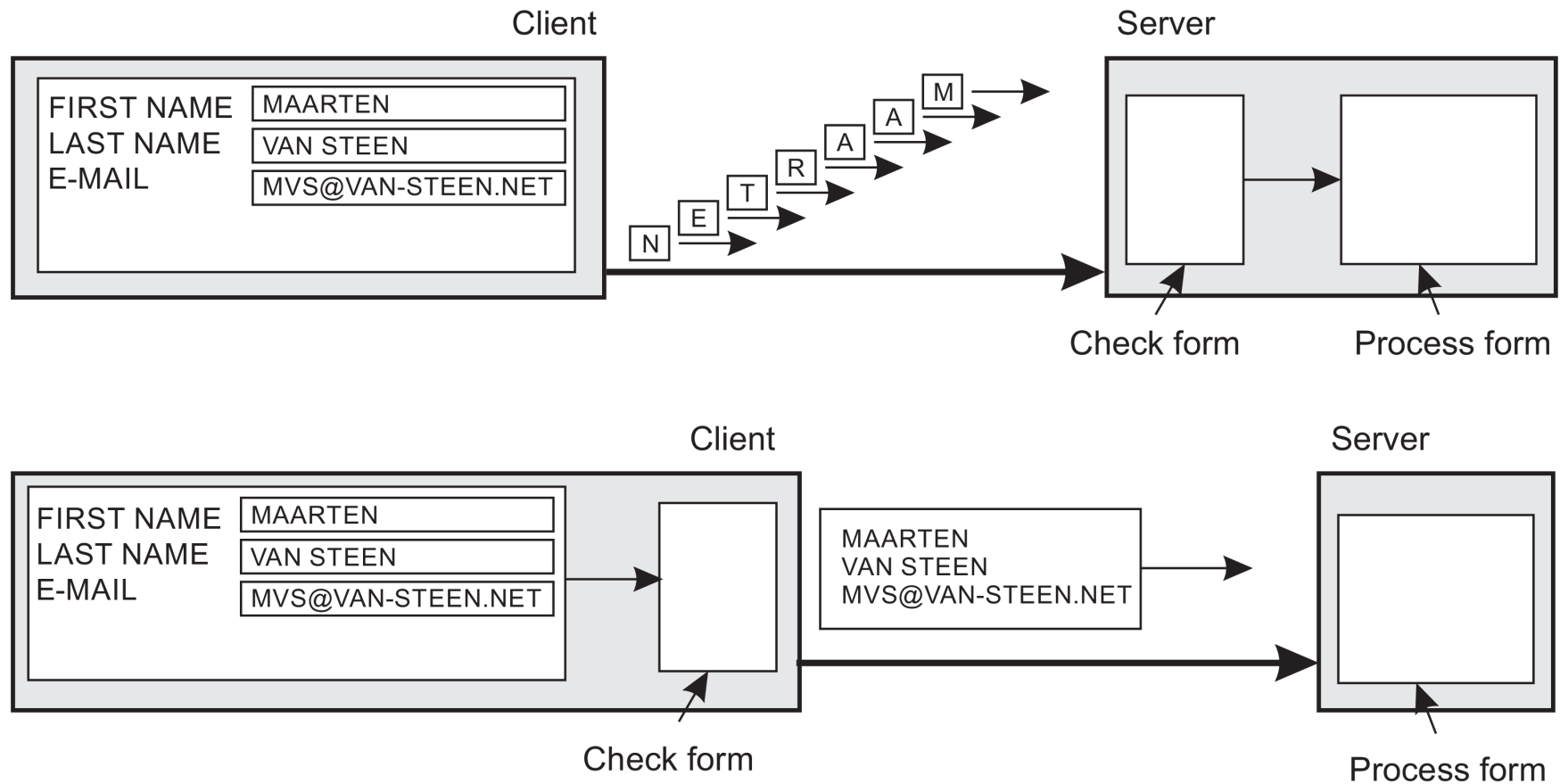
Techniques for scaling

Hide communication latencies

- Make use of **asynchronous communication**
- Have separate handler for incoming response
- **Problem:** not every application fits this model

Techniques for scaling

Facilitate solution by moving computations to client



Techniques for scaling

Partition data and computations across multiple machines

- Move computations to clients (Java applets and scripts)
- Decentralized naming services (DNS)
- Decentralized information systems (WWW)

Techniques for scaling

Replication and caching: Make copies of data available at different machines

- Replicated file servers and databases
- Mirrored Websites
- Web caches (in browsers and proxies)
Store copies of web files on user device or intermediary servers
- File caching (at server and client)

Scaling: The problem with replication

Applying replication is easy, except for one thing

Scaling: The problem with replication

Applying replication is easy, except for one thing

- Having multiple copies (cached or replicated), leads to **inconsistencies**: modifying one copy makes that copy different from the rest.

Scaling: The problem with replication

Applying replication is easy, except for one thing

- Having multiple copies (cached or replicated), leads to **inconsistencies**: modifying one copy makes that copy different from the rest.
- Always keeping copies consistent and in a general way requires **global synchronization** on each modification.

Scaling: The problem with replication

Applying replication is easy, except for one thing

- Having multiple copies (cached or replicated), leads to **inconsistencies**: modifying one copy makes that copy different from the rest.
- Always keeping copies consistent and in a general way requires **global synchronization** on each modification.
- Global synchronization precludes large-scale solutions.

Scaling: The problem with replication

Applying replication is easy, except for one thing

- Having multiple copies (cached or replicated), leads to **inconsistencies**: modifying one copy makes that copy different from the rest.
- Always keeping copies consistent and in a general way requires **global synchronization** on each modification.
- Global synchronization precludes large-scale solutions.

Observation

If we can tolerate inconsistencies, we may reduce the need for global synchronization, but **tolerating inconsistencies is application dependent**.

Example: For some social media apps, inconsistency may be fine (e.g., different users see different messages temporarily)

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex, caused by mistakes that required patching later on. Many **false assumptions** are often made.

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex, caused by mistakes that required patching later on. Many **false assumptions** are often made.

False (and often hidden) assumptions

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex, caused by mistakes that required patching later on. Many **false assumptions** are often made.

False (and often hidden) assumptions

- The network is reliable

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex, caused by mistakes that required patching later on. Many **false assumptions** are often made.

False (and often hidden) assumptions

- The network is reliable
- The network is secure

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex, caused by mistakes that required patching later on. Many **false assumptions** are often made.

False (and often hidden) assumptions

- The network is reliable
- The network is secure
- The network is homogeneous

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex, caused by mistakes that required patching later on. Many **false assumptions** are often made.

False (and often hidden) assumptions

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex, caused by mistakes that required patching later on. Many **false assumptions** are often made.

False (and often hidden) assumptions

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex, caused by mistakes that required patching later on. Many **false assumptions** are often made.

False (and often hidden) assumptions

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex, caused by mistakes that required patching later on. Many **false assumptions** are often made.

False (and often hidden) assumptions

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero

Developing distributed systems: Pitfalls

Observation

Many distributed systems are needlessly complex, caused by mistakes that required patching later on. Many **false assumptions** are often made.

False (and often hidden) assumptions

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator